BRMW: explicação do banco de dados, linha por linha

Documento de apoio ao script `brmw.sql` · MySQL 8.0+ · Projeto de curso técnico

1. Como ler este documento

O banco do BRMW guarda usuários, cursos, turmas, alunos, avaliações, notas, níveis e recompensas. Este documento explica cada parte do script `brmw.sql` na ordem em que ela aparece. Para cada tabela você vê para que ela serve, o que cada coluna faz e o que cada chave ou regra (UNIQUE, KEY, FOREIGN KEY, CHECK) protege.

A ordem das colunas e das regras aqui é a mesma do arquivo, então você pode ler os dois lado a lado.

2. Glossário rápido

- **PRIMARY KEY:** identifica cada linha da tabela de forma única. Não se repete e não pode ser vazia.
- **AUTO_INCREMENT:** o MySQL gera o número sozinho (1, 2, 3...) a cada novo registro.
- **NOT NULL:** a coluna é obrigatória. Sem essa marca, a coluna aceita NULL (vazio).
- **DEFAULT:** valor usado quando você não informa nada ao inserir.
- **UNIQUE KEY:** proíbe valores repetidos, seja numa coluna ou na combinação de várias colunas.
- **KEY (índice):** deixa as buscas por aquela coluna mais rápidas. Não muda as regras dos dados.
- **FOREIGN KEY (chave estrangeira):** liga uma coluna ao `id` de outra tabela. O MySQL recusa um valor que não exista lá.
- **ON UPDATE CASCADE:** se o `id` referenciado mudar, a mudança se propaga para quem o usa.
- **ON UPDATE RESTRICT e ON DELETE RESTRICT:** impedem mudar ou apagar uma linha que ainda é usada por outra tabela. Em todo o BRMW nada é apagado em cascata, para ninguém perder dados sem querer.
- **CHECK:** regra que o valor precisa cumprir, por exemplo `peso > 0`.
- **ENUM:** a coluna só aceita um dos valores listados.
- **INT e INT UNSIGNED:** número inteiro. `UNSIGNED` não aceita negativos.
- **VARCHAR(n) e TEXT:** texto de até `n` caracteres e texto longo, respectivamente.

- **DECIMAL(7,2):** número com 2 casas decimais e até 7 dígitos no total (por exemplo 12345.67).
- **TIMESTAMP:** data e hora. `DEFAULT CURRENT_TIMESTAMP` grava o momento da inserção.
- **ENGINE=InnoDB:** motor de armazenamento do MySQL que suporta chaves estrangeiras e transações. Todas as tabelas usam.
- **Chave composta de proteção:** várias tabelas têm `UNIQUE (id, turma_id)`. Parece redundante com a PRIMARY KEY, mas permite que outras tabelas liguem as duas colunas juntas. Assim o banco impede, por exemplo, usar um critério da turma A numa avaliação da turma B.

3. Cabeçalho e criação do banco

- **Bloco de comentários no topo (linhas que começam com `--`):** o MySQL ignora comentários. Eles documentam a estrutura, as convenções de nomes e as regras gerais.
- **`CREATE DATABASE IF NOT EXISTS brmw`:** cria o banco chamado `brmw`, somente se ele ainda não existir.
- **`CHARACTER SET utf8mb4`:** aceita todos os caracteres, inclusive acentos e emojis.
- **`COLLATE utf8mb4_unicode_ci`:** define como os textos são comparados e ordenados. O `ci` significa que maiúsculas e minúsculas contam como iguais (Ana = ana).
- **`USE brmw;`:** diz ao MySQL que os próximos comandos valem para o banco `brmw`.

4. Mapa dos relacionamentos

Lendo de cima para baixo:

- `usuario` é dono de `curso`, `turma` e `avaliacao`.
- `curso` tem várias `turma`.
- `turma` tem vários `aluno`, `criterio_avaliacao`, `nivel_geral` e `avaliacao`.
- `nivel_geral` tem várias `recompensa`.
- `avaliacao` usa vários critérios por meio de `avaliacao_criterio`.
- `resultado_criterio` guarda a nota de um aluno em um critério de uma avaliação.
- `historico` guarda o percentual e o nível de cada aluno quando a avaliação é finalizada.
- `recompensa_conquistada` registra quais recompensas cada aluno ganhou.
- `observacao` guarda anotações do usuário, gerais ou sobre um aluno.

A ordem de criação das tabelas no script importa: uma tabela só pode referenciar outra que já exista. Por isso `usuario` vem primeiro e `observacao` por último.

5. Tabela usuario

Guarda quem acessa o sistema (professores e administradores).

- `id INT AUTO_INCREMENT PRIMARY KEY`: número único do usuário, gerado automaticamente.
- `tipo ENUM('ADMIN', 'USER') NOT NULL DEFAULT 'USER'`: papel do usuário. Se não for informado, vira `USER`. O backend usa isso para decidir permissões.
- `nome VARCHAR(150) NOT NULL`: nome completo, obrigatório.
- `email VARCHAR(150) NOT NULL`: e-mail usado no login, obrigatório.
- `senha VARCHAR(255) NOT NULL`: guarda o hash da senha (bcrypt ou Argon2), nunca a senha em texto puro. Os 255 caracteres comportam esses hashes.
- `status ENUM('ATIVO', 'INATIVO') NOT NULL DEFAULT 'ATIVO'`: permite desativar um usuário sem apagá-lo.
- `criado_em TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP`: data e hora do cadastro, preenchidas automaticamente.
- `atualizado_em TIMESTAMP ... ON UPDATE CURRENT_TIMESTAMP`: começa com o momento da criação e é atualizado sozinho sempre que a linha for alterada.
- `UNIQUE KEY uk_usuario_email (email)`: dois usuários não podem ter o mesmo e-mail.
- `ENGINE=InnoDB`: motor da tabela (ver glossário).

6. Tabela curso

Guarda os cursos, por exemplo Técnico em Informática. Cada curso pertence ao usuário que o cadastrou.

- `id`: número único do curso.
- `usuario_id INT NOT NULL`: dono do curso, obrigatório.
- `nome VARCHAR(150) NOT NULL`: nome do curso.
- `descricao TEXT`: descrição livre, opcional (aceita NULL).
- `criado_em`: data e hora do cadastro.
- `UNIQUE KEY uk_curso_id_usuario (id, usuario_id)`: chave composta de proteção. Permite que `turma` ligue `curso_id` e `usuario_id` juntos, garantindo que turma e curso tenham o mesmo dono.

- `UNIQUE KEY uk_curso_usuario_nome (usuario_id, nome)`: o mesmo usuário não pode ter dois cursos com o mesmo nome.
- `CONSTRAINT fk_curso_usuario FOREIGN KEY (usuario_id) REFERENCES usuario(id)`: o `usuario_id` precisa existir na tabela `usuario`.
- `ON UPDATE CASCADE`: se o `id` do usuário mudar, o curso acompanha.
- `ON DELETE RESTRICT`: não deixa apagar um usuário que tenha cursos.

7. Tabela turma

Guarda as turmas de um curso em um ano letivo.

- `id`: número único da turma.
- `usuario_id`: dono da turma. Precisa ser o mesmo dono do curso (a regra está na chave estrangeira abaixo).
- `curso_id`: curso ao qual a turma pertence.
- `nome VARCHAR(100) NOT NULL`: nome da turma, por exemplo 3º A.
- `vagas INT UNSIGNED`: número de vagas, opcional. Não aceita negativos.
- `ano_letivo YEAR NOT NULL`: ano da turma, por exemplo 2026.
- `status ENUM('ATIVA', 'INATIVA', 'ENCERRADA') NOT NULL DEFAULT 'ATIVA'`: situação da turma. Começa como ativa.
- `criado_em`: data e hora do cadastro.
- `UNIQUE KEY uk_turma_id_usuario (id, usuario_id)`: chave composta de proteção. Hoje nenhuma outra tabela a usa, então pode ser removida sem efeito.
- `UNIQUE KEY uk_turma_curso_nome_ano (curso_id, nome, ano_letivo)`: impede turma duplicada (mesmo nome, no mesmo curso e no mesmo ano).
- `KEY idx_turma_usuario (usuario_id)`: acelera a busca das turmas de um usuário.
- `KEY idx_turma_curso_usuario (curso_id, usuario_id)`: índice exigido pela chave estrangeira composta abaixo.
- `fk_turma_usuario`: `usuario_id` precisa existir em `usuario`.
- `fk_turma_curso FOREIGN KEY (curso_id, usuario_id) REFERENCES curso(id, usuario_id)`: o curso precisa existir e pertencer ao mesmo usuário da turma.
- `chk_turma_vagas CHECK (vagas IS NULL OR vagas > 0)`: se as vagas forem informadas, precisam ser maiores que zero.

8. Tabela aluno

Cada aluno é cadastrado diretamente em uma turma. Não existe matrícula compartilhada entre turmas.

- `id`: número único do aluno.
- `turma_id INT NOT NULL`: turma do aluno.
- `nome VARCHAR(150) NOT NULL`: nome do aluno.
- `email VARCHAR(150)`: e-mail, opcional.
- `telefone VARCHAR(25)`: telefone, opcional. É texto para aceitar parênteses, traços e o código do país.
- `data_nascimento DATE`: data de nascimento, opcional.
- `codigo_aluno VARCHAR(50)`: código ou matrícula interna, opcional.
- `status ENUM('ATIVO', 'INATIVO') NOT NULL DEFAULT 'ATIVO'`: permite inativar o aluno (por exemplo, quem saiu da turma) sem apagar o histórico.
- `criado_em`: data e hora do cadastro.
- `UNIQUE KEY uk_aluno_id_turma (id, turma_id)`: chave composta de proteção. É usada por resultados, histórico, recompensas e observações para amarrar aluno e turma juntos.
- `UNIQUE KEY uk_aluno_turma_codigo (turma_id, codigo_aluno)`: dentro da mesma turma o código não se repete. Vários alunos podem ficar com código vazio, porque o MySQL trata cada NULL como diferente.
- `fk_aluno_turma`: a turma precisa existir. Não é possível apagar uma turma que tenha alunos.

9. Tabela criterio_avaliacao

Guarda os critérios usados para avaliar os alunos (por exemplo, Trabalho em equipe). Cada critério pertence a uma turma e tem seus limites de nota e seu peso.

- `id`: número único do critério.
- `turma_id`: turma dona do critério.
- `nome VARCHAR(150) NOT NULL`: nome do critério.
- `descricao TEXT`: explicação do critério, opcional.
- `nota_min DECIMAL(7,2) NOT NULL DEFAULT 0`: menor nota possível neste critério.
- `nota_max DECIMAL(7,2) NOT NULL DEFAULT 10`: maior nota possível neste critério.
- `peso DECIMAL(7,2) NOT NULL DEFAULT 1`: importância do critério no cálculo ponderado. Um critério de peso 2 conta o dobro de um de peso 1.

- `status ENUM('ATIVO', 'INATIVO') NOT NULL DEFAULT 'ATIVO'`: permite desativar o critério sem apagá-lo.
- `UNIQUE KEY uk_criterio_avaliacao_id_turma (id, turma_id)`: chave composta de proteção. Permite que `avaliacao_criterio` ligue critério e turma juntos.
- `KEY idx_criterio_avaliacao_turma (turma_id)`: acelera a busca dos critérios de uma turma e serve de índice para a chave estrangeira.
- `fk_criterio_avaliacao_turma`: a turma precisa existir.
- `chk_criterio_avaliacao_notas CHECK (nota_min >= 0 AND nota_max > nota_min)`: a nota mínima não pode ser negativa e a máxima precisa ser maior que a mínima.
- `chk_criterio_avaliacao_peso CHECK (peso > 0)`: o peso precisa ser positivo.

10. Tabela nivel_geral

Guarda as faixas de percentual que classificam o desempenho do aluno na turma. Exemplo: Iniciante de 0 a 49,99 e Avançado de 50 a 100.

- `id`: número único do nível.
- `turma_id`: turma dona do nível.
- `nome VARCHAR(100) NOT NULL`: nome do nível.
- `descricao TEXT`: explicação do nível, opcional.
- `percentual_min DECIMAL(5,2) NOT NULL`: início da faixa, de 0,00 a 100,00.
- `percentual_max DECIMAL(5,2) NOT NULL`: fim da faixa, inclusive.
- `UNIQUE KEY uk_nivel_geral_id_turma (id, turma_id)`: chave composta de proteção. Usada por `recompensa` e `historico`.
- `KEY idx_nivel_geral_turma (turma_id)`: acelera a busca dos níveis de uma turma.
- `fk_nivel_geral_turma`: a turma precisa existir.
- `chk_nivel_geral_faixa`: exige `percentual_min >= 0`, `percentual_max <= 100` e `percentual_max >= percentual_min`.

O banco não impede faixas que se sobreponham ou deixem lacunas (por exemplo, 0 a 49 e 51 a 100). Essa conferência fica no backend.

11. Tabela recompensa

Guarda as recompensas que o aluno ganha ao atingir um nível da turma.

- `id`: número único da recompensa.
- `turma_id`: turma da recompensa.

- `nivel_geral_id`: nível que libera a recompensa.
- `nome VARCHAR(150) NOT NULL`: nome da recompensa.
- `descricao TEXT`: detalhes, opcional.
- `tipo VARCHAR(50)`: categoria em texto livre (por exemplo, Medalha), opcional.
- `status ENUM('ATIVA', 'INATIVA') NOT NULL DEFAULT 'ATIVA'`: só recompensas ativas são concedidas.
- `UNIQUE KEY uk_recompensa_id_turma_nivel (id, turma_id, nivel_geral_id)`: chave composta de proteção com três colunas. Permite que `recompensa_conquistada` confira recompensa, turma e nível de uma vez.
- `KEY idx_recompensa_nivel_turma (nivel_geral_id, turma_id)`: índice da chave estrangeira abaixo.
- `fk_recompensa_nivel_geral FOREIGN KEY (nivel_geral_id, turma_id) REFERENCES nivel_geral(id, turma_id)`: o nível precisa existir e ser da mesma turma da recompensa.

12. Tabela avaliacao

Guarda uma avaliação aplicada a uma turma em uma data, por exemplo a Prova do 1º bimestre.

- `id`: número único da avaliação.
- `turma_id`: turma avaliada.
- `usuario_id`: usuário que criou ou registrou a avaliação. O backend deve autorizar o dono da turma ou um administrador.
- `titulo VARCHAR(150) NOT NULL`: título da avaliação.
- `data_avaliacao DATE NOT NULL`: dia em que a avaliação foi aplicada.
- `observacao TEXT`: anotação livre, opcional.
- `status ENUM('RASCUNHO', 'FINALIZADA') NOT NULL DEFAULT 'RASCUNHO'`: enquanto é rascunho as notas podem ser lançadas e alteradas. Ao finalizar, o backend calcula o histórico e bloqueia novas alterações.
- `criado_em`: data e hora do cadastro.
- `UNIQUE KEY uk_avaliacao_id_turma (id, turma_id)`: chave composta de proteção. Usada por `avaliacao_criterio` e `historico`.
- `KEY idx_avaliacao_turma_data (turma_id, data_avaliacao)`: lista as avaliações de uma turma por data com rapidez e serve de índice para a chave estrangeira da turma.
- `fk_avaliacao_turma`: a turma precisa existir.

- `fk_avaliacao_usuario`: o usuário precisa existir.

13. Tabela avaliacao_criterio

É a tabela de ligação que diz quais critérios fazem parte de cada avaliação. Uma avaliação tem vários critérios, e um critério pode ser usado em várias avaliações.

- `id`: número único do vínculo.
- `avaliacao_id`: avaliação.
- `turma_id`: turma, repetida de propósito. As duas chaves estrangeiras usam essa coluna para garantir que avaliação e critério sejam da mesma turma.
- `criterio_id`: critério usado.
- `UNIQUE KEY uk_avaliacao_criterio_avaliacao_criterio (avaliacao_id, criterio_id)`: o mesmo critério não entra duas vezes na mesma avaliação.
- `UNIQUE KEY uk_avaliacao_criterio_id_turma (id, turma_id)`: chave composta de proteção, usada por `resultado_criterio`.
- `KEY idx_avaliacao_criterio_criterio_turma (criterio_id, turma_id)`: índice da chave estrangeira do critério.
- `fk_avaliacao_criterio_avaliacao FOREIGN KEY (avaliacao_id, turma_id) REFERENCES avaliacao(id, turma_id)`: a avaliação precisa existir e ser dessa turma.
- `fk_avaliacao_criterio_criterio FOREIGN KEY (criterio_id, turma_id) REFERENCES criterio_avaliacao(id, turma_id)`: o critério precisa existir e ser dessa mesma turma. É isso que impede usar um critério de outra turma.

14. Tabela resultado_criterio

Guarda a nota de um aluno em um critério de uma avaliação. É a tabela que mais cresce, com uma linha por aluno por critério.

- `id`: número único do resultado.
- `aluno_id`: aluno avaliado.
- `turma_id`: turma, usada para amarrar aluno e vínculo à mesma turma.
- `avaliacao_criterio_id`: qual critério de qual avaliação está sendo notado.
- `nota DECIMAL(7,2) NOT NULL`: a nota do aluno, obrigatória.
- `comentario TEXT`: comentário do professor sobre a nota, opcional.
- `criado_em`: data e hora do lançamento.

- `UNIQUE KEY uk_resultado_criterio_aluno_avaliacao_criterio (aluno_id, avaliacao_criterio_id)`: cada aluno recebe no máximo uma nota por critério em cada avaliação.
- `KEY idx_resultado_criterio_avaliacao_criterio_turma (avaliacao_criterio_id, turma_id)`: acelera a busca das notas de um critério e serve de índice para a chave estrangeira.
- `fk_resultado_criterio_aluno FOREIGN KEY (aluno_id, turma_id) REFERENCES aluno(id, turma_id)`: o aluno precisa existir e ser da turma informada.
- `fk_resultado_criterio_avaliacao_criterio FOREIGN KEY (avaliacao_criterio_id, turma_id) REFERENCES avaliacao_criterio(id, turma_id)`: o critério da avaliação precisa existir e ser da mesma turma. Juntas, as duas chaves garantem que aluno e critério são da mesma turma.
- `chk_resultado_criterio_nota CHECK (nota >= 0)`: a nota não pode ser negativa. Se a nota está entre `nota_min` e `nota_max` do critério, quem confere é o backend.

15. Tabela historico

Guarda o resultado final de cada aluno em cada avaliação finalizada: o percentual e o nível atingido naquele momento. Assim, mesmo que as faixas mudem depois, fica registrado o que valia na época.

- `id`: número único do registro.
- `aluno_id`: aluno.
- `turma_id`: turma.
- `avaliacao_id`: avaliação finalizada.
- `nivel_geral_id`: nível em que o aluno se enquadrou.
- `percentual DECIMAL(5,2) NOT NULL`: percentual final do aluno, de 0 a 100.
- `data_registro TIMESTAMP ... DEFAULT CURRENT_TIMESTAMP`: quando o registro foi gravado.
- `UNIQUE KEY uk_historico_aluno_avaliacao (aluno_id, avaliacao_id)`: um único registro por aluno em cada avaliação.
- `UNIQUE KEY uk_historico_id_turma_nivel (id, turma_id, nivel_geral_id)`: chave composta de proteção, usada por `recompensa_conquistada`.
- `KEY idx_historico_aluno_data (aluno_id, data_registro)`: acelera a consulta da evolução de um aluno ao longo do tempo.
- `fk_historico_aluno (aluno_id, turma_id)`: o aluno precisa existir e ser da turma.
- `fk_historico_avaliacao (avaliacao_id, turma_id)`: a avaliação precisa existir e ser da mesma turma.

- `fk_historico_nivel_geral (nivel_geral_id, turma_id)`: o nível precisa existir e ser da mesma turma.
- `chk_historico_percentual CHECK (percentual >= 0 AND percentual <= 100)`: o percentual fica sempre entre 0 e 100.

16. Tabela recompensa_conquistada

Registra quais recompensas cada aluno já ganhou e por qual avaliação.

- `id`: número único da conquista.
- `aluno_id`: aluno que ganhou.
- `turma_id`: turma.
- `recompensa_id`: recompensa ganha.
- `nivel_geral_id`: nível que liberou a recompensa.
- `historico_id`: registro do histórico que originou a conquista.
- `data_conquista TIMESTAMP ... DEFAULT CURRENT_TIMESTAMP`: quando o aluno ganhou.
- `UNIQUE KEY uk_recompensa_conquistada_aluno_recompensa (aluno_id, recompensa_id)`: o aluno ganha cada recompensa uma única vez, mesmo que atinja o nível em várias avaliações.
- `KEY idx_recompensa_conquistada_aluno_turma (aluno_id, turma_id)`: acelera a busca das recompensas de um aluno.
- `fk_recompensa_conquistada_aluno (aluno_id, turma_id)`: o aluno precisa existir e ser da turma.
- `fk_recompensa_conquistada_recompensa (recompensa_id, turma_id, nivel_geral_id)`: a recompensa precisa existir, ser da turma e estar ligada ao nível informado.
- `fk_recompensa_conquistada_historico (historico_id, turma_id, nivel_geral_id)`: o registro do histórico precisa existir, ser da turma e ter o mesmo nível. Ou seja, não dá para registrar uma recompensa de um nível que o aluno não atingiu.

17. Tabela observacao

Guarda anotações do usuário. Elas podem ser gerais ou ligadas a um aluno de uma turma.

- `id`: número único da observação.
- `usuario_id INT NOT NULL`: quem escreveu a observação.

- `aluno_id INT NULL`: aluno ao qual a observação se refere. Vazio quando for uma observação geral.
- `turma_id INT NULL`: turma do aluno. Vazia quando for uma observação geral.
- `titulo VARCHAR(150) NOT NULL`: título curto.
- `descricao TEXT NOT NULL`: texto da observação.
- `criado_em`: data e hora do cadastro.
- `KEY idx_observacao_usuario (usuario_id)`: acelera a busca das observações de um usuário.
- `KEY idx_observacao_aluno_turma (aluno_id, turma_id)`: acelera a busca das observações de um aluno.
- `fk_observacao_usuario`: o usuário precisa existir.
- `fk_observacao_aluno FOREIGN KEY (aluno_id, turma_id) REFERENCES aluno(id, turma_id)`: se houver aluno, ele precisa existir e ser da turma informada. Aqui a regra é `ON UPDATE RESTRICT` e não `CASCADE`, porque o MySQL não permite usar em um CHECK colunas de uma chave estrangeira com CASCADE (erro 3823).
- `chk_observacao_vinculo`: ou `aluno_id` e `turma_id` são os dois vazios (observação geral) ou os dois preenchidos. Sem esse CHECK seria possível gravar só um dos dois, porque o MySQL não confere uma chave estrangeira composta quando uma das colunas é NULL.

18. Fim do script e o que fica no JavaScript

O script termina com um comentário de encerramento. O banco não tem triggers nem procedures, então estas regras precisam ser feitas no backend:

- **Nota dentro dos limites:** conferir se `nota_min <= nota <= nota_max` do critério antes de salvar.
- **Avaliação finalizada:** recusar qualquer alteração de notas quando `avaliacao.status = 'FINALIZADA'`.
- **Notas completas:** antes de finalizar, conferir se todo aluno ativo tem nota em todos os critérios.
- **Faixas de nível:** conferir que cobrem de 0 a 100 sem lacunas nem sobreposições.
- **Cálculo do percentual:** para cada aluno, somar `(nota - nota_min) / (nota_max - nota_min) * peso` de todos os critérios, dividir pela soma dos pesos e multiplicar por 100. Arredonde para 2 casas antes de procurar a faixa do nível.
- **Gravação:** salvar o `historico`, conceder as `recompensa_conquistada` e marcar a avaliação como `FINALIZADA`, tudo numa única transação. Se algo falhar, desfazer tudo (rollback).

- **Permissões:** validar se o usuário é o dono da turma ou um `ADMIN`.

Exemplo do cálculo: o critério A (0 a 10, peso 2) recebeu nota 8 e o critério B (0 a 10, peso 1) recebeu nota 6. O percentual é (0,8 × 2 + 0,6 × 1) ÷ 3 × 100 = 73,33%. Esse valor é comparado com as faixas de `nivel_geral` para achar o nível do aluno.